# 연산량이 아닌 병목으로 설계하기: 입력 대역에 묶인 소형 FPGA 정수 가속기에서의 CNN 영상 잡음 제거

*(영문 제목: Designing to the Bottleneck, Not the FLOPs: CNN Image Denoising on an Input-Bandwidth-Bound Small-FPGA Integer Accelerator)*

(저자 이름), (소속: 학과, 대학교), (이메일)

**초안 메모 — 제출 전에 확정할 것**

- 2026-10-14 판. 6개 관점의 심사 패널과 24개 지적의 반박 검증을 거쳐 고쳤다. 모든 수치는 저장소의 `denoise/export/*/int_eval.json`, `denoise/board_runs/*.json`, `denoise/bm3d_protocol.json`, `denoise/dncnn_s_protocol.json`, `denoise/ssim_eval.json`에서 나온 것이며, `python denoise/results_table.py`가 표 I을 다시 만든다.
- 영문 4쪽(IEEE 2단)으로 옮길 때 줄일 순서: 표 II의 층별 행(모델당 한 행으로) → V절 U-Net 문단 → 참고문헌 주석. IV절(검증)은 줄이지 않는다.
- 사사(acknowledgment)의 AI 도구 사용 고지는 저자가 결정한다(아래 문장은 안).
- 2026-10-15 판: 감사(숫자 대조, 지적 해소, 문서 간 일관성, 과장 표현)에서 확인된 오류를 고쳤다. 〔재실험〕 표시는 같은 120 epoch으로 다시 학습 중인 ReLU·평균 결합·maxout 비교의 결과로 바꿀 자리다.
- 2026-10-16 판: 속도 모델을 같은 예산(120 epoch + QAT 15 epoch, 시드 1)으로 학습한 `dn_d10_c32_us_mo_fx_qat`로 바꿨다(보드 0 mismatches, 44,254,293 사이클). 같은 예산의 ReLU+QAT 망도 보드에서 실행해(43,811,412 사이클, 668.5/px, 모델과 같음) 보드 검증 모델이 일곱 개가 됐다. B-32 행은 시드 1 값이며, maxout 대 ReLU의 차이는 시드 2·3 결과로 정한다(규칙은 `ABSTRACT.md`).
- 2026-10-17 (늦은 밤) 판: W=4 비트스트림(100 MHz에서 WNS +0.130 ns, LUT 21%, BRAM 51%)으로 네 망을 보드에서 돌렸다(`denoise/board_runs/*_05_w4.json`, `denoise/compare_w4.py` → `w4_measured.json`). 모든 층 불일치 0, PSNR은 1값과 같고, 총 사이클은 비트스트림을 만들기 전에 커밋한 예측(7e81f6b, `w4_prediction.json`)보다 0.0009-0.0044% 많았다(층마다 9-164 사이클, 채움·비움). IN_STALL은 네 망 합계 12 사이클(B-32 ReLU conv5), OUT_STALL 0. 1값·4값 보드 실행과 RTL 표본 각각이 계수와 보인 차이(IN_STALL을 뺀 값)는 은닉층 29개 모두에서 min(창 적재 ⌈K/w⌉+1, 마지막 창의 계산) + 19 사이클이고 마지막 층은 + COUT + 3이다(`denoise/w4_gaps.py` → `w4_gaps.json`; 실행 뒤에 읽어 낸 규칙). B ÷ A 1.194(1값 0.746), maxout ÷ ReLU 2.482(1값 1.010). 초록, I절, II절, 표 II·III, IV절, V절, 결론의 4값 문장을 '보드 측정 전'에서 측정으로 바꿨다. 2값과 라인 버퍼는 예측 그대로다.
- 2026-10-17 (밤) 판: 사이클당 4값을 받는 코어 변형(W=4)을 만들어, 네 망의 모든 층에서 실제 창으로 RTL 시뮬레이션했다(보드 측정 전; `denoise/sim_w4_layers.py` → `w4_rtl_sim.json`). 표 III의 sparse 값은 보드에서 잰 활성값 밀도에 뱅크 불균형 1.0-1.3을 가정한 추정이었는데, 05.png의 모든 창을 센 값으로 바꿨다(`denoise/wider_ports.py`). 옛 추정은 그룹당 비용을 ⌈K/4⌉로 자르지 않았고, 두 망에 같은 불균형을 가정해 maxout ÷ ReLU를 4값에서 2.59-2.79, 라인 버퍼에서 2.85-2.88로 높게 잡았다(센 값 2.48, 2.61). 망별 값과 B ÷ A는 옛 범위 안이고, 정성적 결론(2값부터 합치기가 공짜가 아니고, 4값부터 B가 A보다 느림)은 그대로다. dense 상한도 창당 +1을 적재 항에만 붙이도록 고쳤다(RTL에서 계산에 묶

인 창에는 +1이 없다). 4값에서 A 833 → 828, B 1,209 → 1,207, B-32 ReLU 319 → 317 사이클/px처럼 1% 이내로 바뀐다. 계산 항의 VGG11 대조 문장은 실제 창의 RTL 대조로 대신했다(README에는 남아 있다).

- 2026-10-17 (뒤) 판: 적대적 심사를 반영해 II절의 병목 설명을 고쳤다. 처음에 '래퍼의 설계'라고 적은 것은 부정확했다(코어의 창 입력 포트 자체가 사이클당 1값). 계산 항의 RTL 시뮬레이션 대조는 CIFAR-10이 아니라 VGG11 `features.3` 창이다.
- 2026-10-17 판: 시드 2·3(다른 PC의 GPU)과 45채널 ReLU 망, 평균 결합 시드 2가 나왔다. 정수(QAT) Set12 / BSD68은 ReLU 29.62 / 28.68, 29.94 / 28.91, 29.89 / 28.88, maxout 30.00 / 28.94, 29.91 / 28.89, 29.99 / 28.95로 시드 2에서 순서가 뒤집혀 (−0.03 / −0.02 dB), `ABSTRACT.md`의 규칙대로 (c)를 적용했다. 즉 maxout이 화질을 올린다는 주장을 뺐다. 평균 결합 시드 1의 QAT는 아직 옛 PC에 있다. 시드 2·3은 커밋 261952a 이전의 학습 코드, 즉 PyTorch의 GPU 기본 설정(GPU가 지원하면 합성곱에 TF32)으로 학습됐다(그 커밋이 넣은 fp32 설정이 아님).

---

## 초록


효율적인 신경망은 보통 곱셈 수를 최소화하도록 설계되지만, 소형 가속기에서는 실행 시간이 다른 것으로 정해질 수 있다. 본 연구는 소형 FPGA용 정수 CNN 가속기(Zynq XC7Z020: 곱셈기 64개, 단일 비트스트림, 7비트 입력, ReLU 출력)에서 가우시안 잡음(σ=25)을 제거하며, 측정된 병목이 망 설계를 어떻게 바꾸는지 보인다. 사이클 모델(설계 단계에서 1.7% 이내, 이전 보드 실행에서 관찰한 보정 두 개를 넣으면 0.003% 이내)은 PC가 만든 창을 사이클당 7비트 값 하나씩 받는 이 구현이 입력 포트에 묶여 있음을 보인다. 즉 한 층의 비용은 창당 K 사이클이며(K는 입력 값의 수), 출력 채널 수는 곱셈기 수(64)까지 비용에 영향을 주지 않는다. 그래서 7층 DnCNN 형태의 기준 망은 곱셈기의 41%만 쓴다. 여기서 연산량(FLOP) 기준으로는 낭비로 보이는 두 가지 선택이 나온다. 2×2 space-to-depth 입력은 10층 64채널 망(파라미터 6.4배, 곱셈 1.6배)을 기준 망보다 25% 적은 사이클에 돌리게 하며, Set12에서 0.3 dB(BSD68 0.2 dB) 높다. 은닉층의 출력을 2배로 늘려 두 개씩 합치면(maxout) 놀고 있던 곱셈기를 쓸 수 있다. 보드에서 출력 64채널 층은 창당 289 사이클로 출력 32채널 층과 같았고, 그래서 망은 파라미터가 2배가 되는 대신 사이클은 1%만 늘었다(675 대 669). 그러나 같은 학습 예산에서 같은 폭의 ReLU 망보다 화질이 높은지는 시드 세 개 중 하나에서 뒤집혀(시드 평균 +0.15 / +0.10 dB, 시드별 Set12 +0.37, −0.03, +0.10 dB), 결과 전에 정한 규칙에 따라 주장하지 않는다. 보드 연산을 재현한 양자화 인지 학습은 ReLU·maxout 정수 모델을 모두 float의 0.04 dB 안으로 가져왔다. 화질 우선 정수 모델은 30.12 / 29.02 dB로, 같은 클리핑 7비트 입력에서 측정한 BM3D(29.95 / 28.37 dB, 클리핑 없이는 29.98 / 28.62 dB)보다 높다. 속도 우선 모델은 기준 망보다 2.63배 적은 코어 사이클로 30.00 / 28.94 dB다(시드 세 개 평균 29.97 / 28.92 dB). 합치기 없는 같은 32채널 망도 시드 평균으로 기준 망과 같은 화질(29.82 / 28.82 dB)을 2.66배 적은 사이클에 낸다. 보드에서 실행한 일곱 모델마다 Set12 한 장을 끝까지 실행했고, 모든 층이 정수 모델과 비트 단위로 일치했다. 입력이 사이클당 2값이면 같은 모델은 합치기가 더는 공짜가 아니라고 예측하고, 4값이거나 칩 안에 라인 버퍼를 두면 망 B도 기준 망보다 느려진다고 예측한다. 사이클당 4값을 받는 코어 변형을 만들어 보드에서 네 망을 돌리자, 비트스트림을 만들기 전에 커밋한 예측과 총 사이클이 0.005% 안에서 맞았고 B는 A의 1.19배, 합치기는 ReLU 망의 2.48배가 되었다. 고를 망은 측정한 병목에 따라 달라진다.



**주제어** — 영상 잡음 제거, CNN 가속기, FPGA, 하드웨어 인지 신경망 설계, maxout, 양자화 인지 학습


---

## I. 서론

DnCNN [2] 같은 CNN 잡음 제거기는 고전 기법 BM3D [1]보다 화질이 좋지만 픽셀마다 수만 번의 곱셈이 필요하다. 가속기에 올릴 망을 고를 때는 보통 곱셈 수(FLOPs)나 파라미터 수를 비용으로 삼는다. 그러나 연산량과 실제 지연이 비례하지 않는다는 것은 알려져 있다. Roofline 모델 [12]은 연산 강도가 낮은 커널은 연산기가 아니라 대역폭에 묶인다고 설명하고, ShuffleNet V2 [9]는 GPU 실측 속도로 설계 지침을 세웠으며, EfficientNet-X [13]는 TPU/GPU에서 FLOPs와 지연의 비비례를 분석해 space-to-depth를 가속기 활용도를 위해 썼고, ABPN [14]은 모바일 NPU에서 연산자별 지연을 재어 INT8 초해상도 망을 설계했다. FPGA CNN 가속기는 칩 안의 파이프라인과 라인 버퍼로 메모리 대역폭 병목을 줄이는 방향으로 설계되어 왔다(PipeCNN [21] 등). 영상 복원에서는 eCNN/ERNet [15]이 DRAM 대역폭 제약에 맞춰 복원망을 공동 설계했고, 경량 CNN 잡음 제거기를 Zynq 계열 FPGA에 올린 보고 [18], [19]와 벤더 DPU에서 INT8 양자화 인지 학습으로 잡음을 제거한 ReTiDe [20]가 있다. 이들은 GPU, TPU, NPU, 벤더 DPU의 측정 지연이나 DRAM 대역폭을 다뤘다. 가장 가까운 연구로, HALP [25]는 GPU에서 잰 계단 모양의 지연(채널 수에 따라 계단처럼 바뀜)에 맞춰 채널을 가지치기하고, DRACO [26]는 데이터 흐름이 아니라 망을 바꿔 메모리에 묶인 망이 고정된 시스톨릭 배열의 PE를 더 쓰게 했다. 본 연구는 주어진 입력 병목이 어떤 망을 유리하게 만드는지를 정확한 사이클 모델로 정하고 보드에서 비트 단위로 확인한다는 점이 다르다.

본 연구는 같은 원리를 더 작은 규모에서, 사이클 단위로 다룬다. 가속기는 Zynq XC7Z020에 올린 정수 CNN 코어로, 곱셈기가 64개이고 하나의 비트스트림이 층마다 형상만 레지스터로 바꿔 모든 층을 실행한다. 입력은 7비트 부호 없는 값, 모든 출력에는 ReLU가 걸린다. 현재 구현의 입력 포트는 사이클당 값 하나를 받으므로, 창 하나의 비용은 곱셈 수가 아니라 **스트리밍되는 값의 수 K**로 정해지고, 출력 채널은 곱셈기 수(64)까지 비용이 없다. 이 성질을 실측으로 확정한 사이클 모델에서 설계를 출발한다. 이 코어는 PC CPU보다 느린 실험용 플랫폼이며(V절), 목적은 처리량이 아니라 사이클을 정확히 따질 수 있는 환경에서 망 선택의 근거를 보이는 것이다.

기여는 다음과 같다.

1. 이 코어에서 **공짜인 것과 비용인 것을 가르는 사이클 모델**. 설계 단계 모델은 보드 실측과 1.7% 이내, 이전 보드 실행에서 본 보정 두 개를 넣으면 0.003% 이내로 맞는다. 공짜 집합은 출력 채널($\leq 64$), 커널 모양, 리샘플링, host 쪽 비선형과 양자화기이고, 비용은 K다.
2. 그 모델이 가리키는, **연산량 기준으로는 손해인 두 선택**을 같은 사이클 축에서 비교한 표(iso-cycle 비교). space-to-depth는 6.4배 큰 망을 25% 적은 사이클에 넣고, 은닉층 출력을 2배로 늘려 두 개씩 합치는 설계(maxout)는 은닉층의 사이클을 바꾸지 않은 채(보드 실측 289 사이클/창, ReLU 층과 동일) 파라미터를 2배로 늘린다. 같은 폭의 ReLU 망, 평균으로 합치는 대조군, 같은 파라미터의 45채널 ReLU 망과 비교했고(ReLU·maxout은 시드 세 개), 같은 폭의 ReLU 망에 대한 화질 이득은 시드마다 엇갈렸다.
3. **보드에서의 비트 단위 검증.** 보드 연산을 재현한 양자화 인지 학습(방법은 기존 관행, III절)으로 만든 정수 모델은 같은 입력 조건의 BM3D를 넘고(30.12 / 29.02 dB), 속도 우선 모델은 기준 망보다 2.63배 적은 사이클로 BM3D 수준이다(30.00 / 28.94 dB). 보드에서 실행한 일곱 모델은 모든 층이 정수 모델과 비트 단위로 일치했다.

space-to-depth [3], [13], maxout [4], [16], [17], 양자화 인지 학습 [5] 자체는 알려진 기법이다. 본 연구의 주장은 기법이 아니라 **선택의 근거**다. 측정된 병목이 설계 규칙을 뒤집는다는 것을 작은 코어에서 사이클 단위로, 비트 단위로 검증한 사례 연구다. 결론은 PC가 만든 창을 사이클당 값 하나로 받는 입력 구조에 대한 것이며, 입력 포트가 사이클당 4값을 받거나(보드 실측) 칩 안에 라인 버퍼를 두면(예측) maxout은 더는 공짜가 아니고 망 B는 기준 망보다 느려진다(II절).

## II. 가속기와 사이클 모델

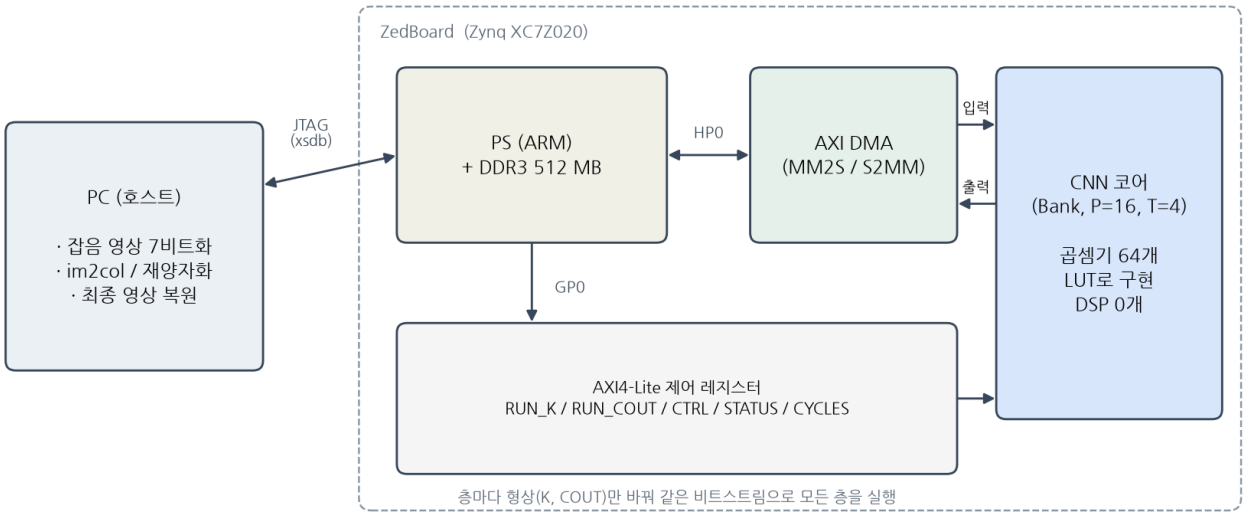



*그림 1. 시스템 구성. PC가 창(im2col)과 층 간 재양자화를 맡고, PL의 코어가 모든 합성곱을 계산한다.*

**연산.** 코어는 창 하나(활성값 K개)를 받아 COUT개 채널의 $\mathrm{ReLU}(b + \Sigma\ w \cdot x)$를 계산한다. 가중치 INT8, 누산 INT32, 활성값 7비트. 곱셈기는 출력 채널 P=16개 × 탭 뱅크 T=4개 = 64개이다. 런타임 형상은 $K \le 1152$, $COUT \le 128$이고, 같은 비트스트림으로 CIFAR-10 분류망과 본 잡음 제거망을 모두 실행했다. 설계는 XC7Z020의 LUT 21%, BRAM 51%를 쓴다. DSP는 0개다. 보드 빌드에서 DSP 사용을 제한하지 않았는데 Vivado가 8비트 곱셈을 모두 LUT로 구현했고(관측이며 보장되는 성질은 아니다), 그 결과 DSP 220개는 다른 용도로 남는다. 타이밍은 100 MHz에서 닫혔고(WNS +0.221 ns), 측정은 PS 초기화 기본값인 50 MHz에서 했으며 한 모델은 100 MHz에서도 돌려 사이클 수가 같음을 확인했다(표 II).

**사이클 모델.** 입력 포트는 사이클당 값 하나를 받고, 출력 채널은 P개씩 묶어 묶음마다 $\lceil K/T \rceil$ 사이클에 처리한다. 다음 창을 받는 동안 현재 창을 계산하므로 창 하나의 비용은

$C = \max(\ K,\ \lceil COUT/P \rceil \cdot \lceil K/T \rceil\ )$. (사이클/창)

K가 앞서면 코어는 **입력 대역에 묶인다**. 이때 $COUT \le P{\cdot}T = 64$인 한 출력 채널은 사이클을 바꾸지 않는다. 예를 들어 K=288에서 COUT 32와 64는 둘 다 288 사이클이고, 128이면 576이다. 이 식은 dense 모드의 상한이다. 보드 실행은 모두 sparse 모드였는데, 이 모드는 0인 활성값의 곱셈만 건너뛰어 연산 항을 줄이고 입력 항 K는 그대로이므로, 입력에 묶인 층의 사이클은 두 모드에서 같다. dense 모드의 COUT > 64는 보드에서 재지 않았다. 같은 비트스트림의 CIFAR-10 실행에서 K=576, COUT=128 층은 sparse 모드(활성값의 65%가 0)로 591.5 사이클/창이었다. 이런 해석적 비용 모델은 FPGA 가속기 설계에서 흔히 쓰인다 [8].

**실측 보정.** 기준 7층 망의 보드 실행에서 모든 층이 이 값보다 창당 1 사이클 많았고, 창당 최소 COUT 사이클이 걸렸다(코어가 출력을 사이클당 하나씩 내보내므로; 같은 현상을 CIFAR-10 실행에서도 보았다). 두 보정은 매개변수가 없는 규칙이다. 창당 +1은 CIFAR-10 실행과 기준 망에서 보였고, 최소 COUT 규칙은 기준 망(첫 층 K=9, COUT=32 → 32 사이클)만으로는 "최소 32 사이클"과 구별되지 않아 B의 첫 층(K=36, COUT=64 → 64 사이클)에서 확정했다. 두 보정을 넣은 모델 $C' = \max(K + 1, \lceil COUT/P \rceil \cdot \lceil K/T \rceil, COUT)$는(+1은 창 적재에 붙고 계산에 묶인 창에는 붙지 않는다) 네 망의 총 사이클과 0.003% 안에서 맞고(표 II), 보정을 코드로 옮긴 뒤 처음 잰 B-32 ReLU 망은 미리 0.0014% 안에서 예측했다. 창당 1 사이클(289분의 1)은 RTL에 남은 비효율이며, 고치지 않고 모델에 넣었다. 보정 전 모델의 오차는 네 망에서 0.4-1.6%다.

**병목의 출처.** 입력이 사이클당 값 하나인 것은 FPGA의 물리 제약이 아니라 이 설계의 선택이다. 코어의 창 입력 포트가 사이클당 7비트 값 하나를 받고(`rtl/banked_window_mac.sv`의 `s_data[6:0]`), AXI-Stream 래퍼는 32비트 전송 단위(beat)에 실려 오는 값 4개를 그에 맞춰 1개/사이클로 풀어 넣는다. 래퍼만이 아니라 코어의 적재 경로도 바꿔야 포트가 넓어진다. 본 연구의 주된 결과는 CIFAR-10용으로 만든 코어를 바꾸지 않고 주어진 것으로 다룬다(이미 있는 가속기를 쓰는 사용자의 입장). 일반화하면 추가 사이클 없이 늘릴 수 있는 출력 채널은 대략 곱셈기 수를 입력 폭으로 나눈 값(P·T/w: 사이클당 1값이면 64, 2값이면 32, 4값이면 16)까지다. 이 규칙을 시험하려고 포트가 beat 하나(w = 4값)를 한 사이클에 받는 변형도 만들었다. 탭 4r..4r+3은 네 뱅크의 같은 행에 하나씩 들어가므로 뱅크마다 사이클당 튜플 하나라는 구조는 그대로이고, 창 적재가 K+1에서 ⌈K/4⌉+1 사이클로 준다(창 길이가 4의 배수가 아니면 PC가 0으로 채운다). 창마다 적재(⌈K/w⌉+1), 계산(⌈COUT/P⌉ × 뱅크별 0 아닌 탭 수의 최댓값), 출력(COUT) 중 가장 긴 것으로 05.png의 모든 창을 세면(보드와 같은 sparse 모드), maxout은 2값에서 ReLU 망의 1.55배, 4값에서 2.48배가 되고, B는 2값에서 아직 A의 0.82배지만 4값에서 1.19배로 A보다 느려진다(표 III). 4값 변형을 보드 비트스트림과 같은 설정(K ≤ 1152, COUT ≤ 128, P = 16, T = 4)으로 RTL 시뮬레이션해 네 망의 모든 층(37개)에 실제 창 512개씩을 넣자, 출력은 모두 정수 모델과 같았고 사이클은 이 계수와 층마다 10-164 사이클 안에서 맞았다. 그 차이는 실행마다 한 번 붙는 파이프라인의 채움·비움으로, 창 수와 무관하다(무엇인지는 아래 보드 실측에서). 이 시뮬레이션은 DMA가 매 사이클 beat를 대 주는 입력을 가정했다. 같은 창을 1값으로 돌린 RTL도 같은 방식으로 맞았다(`denoise/sim_w4_layers.py`). 이어서 이 변형을 보드에서 돌렸다(표 II). 타이밍은 100 MHz에서 닫혔고(WNS +0.130 ns), 설계는 1값 빌드처럼 LUT 21%, BRAM 51%를 쓴다. 네 망의 모든 층이 정수 모델과 비트 단위로 일치했고(05.png PSNR도 1값과 같다), 총 사이클은 비트스트림을 만들기 전에 커밋한 예측(커밋 7e81f6b의 `denoise/w4_prediction.json`)보다 0.0009-0.0044% 많았다. 층마다의 차이 9-164 사이클도 같은 채움·비움이다. 층의 창은 모두 같은 시간에 적재되고 각 창의 계산이 다음 창의 적재와 겹치므로, 창마다 max(적재, 계산)을 더한 계수는 마지막 창의 적재와 계산 중 짧은 쪽을 한 번 빠뜨린다. 실행 뒤에 읽어 보니 은닉층 29개 모두에서 차이(IN_STALL을 뺀 값)는 min(창 적재 ⌈K/w⌉+1, 마지막 창의 계산) + 19 사이클이었고 마지막 층은 + COUT + 3이었으며, 1값·4값 보드 실행과 RTL 표본에서 모두 그랬다(`denoise/w4_gaps.py`). 보드의 차이가 RTL 표본(K=288에서 91-92, K=576에서 164)보다 작은 은닉층과 마지막 층은, A와 B-32 ReLU의 은닉층처럼 영상 모서리의 마지막 창이 0이 많아 계산이 적재보다 짧은 층이다. DMA가 입력을 늦춘 것은 네 망을 합쳐 12 사이클(B-32 ReLU의 conv5)뿐이었고, 이 12 사이클은 보정하지 않고 그 층의 사이클에 그대로 두었다(그 층의 차이 77 = 65 + 12). 즉 B ÷ A는 1값의 0.75에서 1.19로, maxout ÷ ReLU는 1.01에서 2.48로 바뀌었다(`denoise/compare_w4.py` → `w4_measured.json`). 칩 안에 라인 버퍼를 두는 일반적인 FPGA 가속기 구조 [21]에서는 창마다 새 입력 픽셀 하나의 C_in개 값만 들어오므로, 같은 계수로 maxout은 ReLU의 2.61배, B는 A의 1.23배가 된다(dense 상한으로는 A 827 대 B 1,206 사이클/px; 예측이며 그런 하드웨어는 만들지 않았다). 전달하려는 것은 규칙 자체가 아니라, 측정한 병목에 맞춰 망을 고르면 연산량 기준의 선택이 뒤집힌다는 방법이다.

**공짜인 것, 그리고 host의 몫.** 창을 PC가 만들기 때문에 창의 모양(팽창, 2×2 위상 분할, 임의의 탭 배치)은 코어 사이클이 아니다. 층 사이의 재양자화도 PC가 하므로 그 자리에서 하는 값별 연산(채널 쌍의 max, 채널의 공간 배치)도 코어 사이클이 아니다. 코어 사이클에는 오직 K가 든다. host의 몫은 0이 아니다. 256×256 영상 한 장에 im2col, 재양자화, maxout의 max를 합쳐 0.07-0.13 s이며(4코어 PC, numpy; `scripts/time_host.py`, 대부분이 재양자화), 50 MHz 코어 시간의 5-9%다. 은닉층에서는 창당 host 연산(COUT + COUT/2)이 K보다 작으므로 스트리밍 단으로 하드웨어화해도 입력 한계 아래에 숨는다(첫 층은 예외). 표 II의 벽시계 시간이 이보다 훨씬 긴 것은 JTAG 전송과 보드 쪽 검증 오라클 때문이다.

## III. 하드웨어에 맞춘 망 설계

**제약과 기본 망 (A).** DnCNN을 줄인 7층 × 32채널 3×3 망이다(배치 정규화는 합성곱에 합친다). 입력은 [0, 1]로 자르고 7비트로 양자화한다(학습과 평가에서 동일). 모든 출력에 ReLU가 걸리므로 DnCNN처럼 잡음(부호 있음)

을 예측할 수 없다. 그래서 마지막 층이 **깨끗한 영상을 직접 출력**하고, 잡음 영상을 마지막 층에 한 채널 더 입력(skip)해서 "입력을 베끼고 고치는" 경로를 준다. 부호를 둘로 나눠 잡음을 예측하는 변형((w, b), (−w, −b) 두 채널)도 시험했지만 정수 변환 손실이 0.30 dB로 직접 출력 망(0.12 dB)보다 커서 쓰지 않았다(60 epoch 모델 기준).

**병목 분석.** A의 중간 층(K=288, COUT=32)은 계산에 ⌈32/16⌉·72 = 144 사이클이면 되지만 입력을 받는 데 288 사이클이 걸린다. 망 전체로 곱셈기 활용률(dense 곱셈 수 / (64 × 사이클))은 **41%**다. 현재 포트 폭에서는 곱셈기를 늘려도 빨라지지 않고, 줄여도 느려지지 않는다. 설계 방향은 둘이다. K를 줄이거나, 노는 곱셈기를 공짜로 쓰거나.

**space-to-depth (B).** FFDNet [3]처럼 영상을 2×2 위상으로 나눠 반쪽 해상도 4채널로 처리한다. 창 수가 1/4이 되므로, 같은 사이클 예산 안에 채널 64, 깊이 10층의 망이 들어간다. 중간 층은 K=576, COUT=64로 ⌈64/16⌉·144 = 576 = K: 입력 대역과 연산이 정확히 균형을 이룬다(망 전체 활용률 88%). 파라미터는 A의 6.4배, 곱셈은 1.6배인데 사이클은 25% 적다. 이 비교에서 바뀐 것은 리샘플링만이 아니라 깊이·폭·파라미터이므로, 아래 표의 B 대 A 차이는 "space-to-depth 덕분에 더 큰 망이 더 적은 사이클에 들어간다"로 읽어야 한다.

**공짜 출력 채널을 두 개씩 합치기 (B-32+maxout).** B의 채널을 32로 줄이면 중간 층이 K=288로 돌아와 사이클이 B의 절반이 되지만, 다시 출력 채널 64까지가 공짜가 된다. 이 공짜 출력으로 망의 용량을 늘린다. 은닉층이 채널마다 후보 두 개를 계산하고(출력 64), PC가 재양자화할 때 큰 쪽만 다음 층으로 넘긴다(입력 32, K=288 유지). 합치기가 필요한 이유는 다음 층이다. 출력 64개를 그대로 넘기면 다음 층의 K가 576이 되어 사이클이 2배가 된다. 두 개씩 합쳐 32개로 넘기면 출력 쪽의 공짜 용량은 쓰고 비용이 드는 입력 쪽은 그대로 둔다. 같은 파라미터를 ReLU만으로 얻으려면 은닉층이 45채널이어야 하는데(파라미터 149,593, maxout 망 151,636; 바이어스 포함), 그러면 K가 405로 늘어 사이클 모델로 934 사이클/px, maxout 망보다 38% 비싸다(V절에서 학습해 비교한다).

$$\max(\mathrm{ReLU}(a), \mathrm{ReLU}(b)) = \mathrm{ReLU}(\max(a, b)).$$

이것이 maxout [4] (Max-Feature-Map [10] 형태)이다. 복원망에서 maxout의 효과는 망 폭에 따라 다르다. Choi와 Kim [16]은 초해상도에서 maxout으로 필터 수를 절반으로 줄여도 화질을 유지했고, MTLU [17]는 4× space-to-depth 잡음 제거망(64채널, σ=50)에서 maxout이 ReLU보다 약 0.04 dB 낮다고 보고했다. 이 코어의 특징은 그 효과를 사이클 대가 없이 시험할 수 있다는 점이다. max 자체의 효과와 늘어난 용량의 효과를 가르려고, 같은 64 출력을 재양자화 뒤 평균((a+b+1)>>1)으로 합치는 대조군을 같은 조건으로 학습했다(V절). 평균 결합은 다음 층 가중치가 쌍마다 묶인 64채널 ReLU 층과 같아서, 파라미터와 사이클이 maxout과 같고 max만 없다. 일반 하드웨어에서 maxout은 층의 곱셈을 2배로 늘리지만, 여기서는 다음 층의 K가 그대로이므로 $K \geq \mathrm{COUT}$인 은닉층의 사이클이 그대로다(첫 층은 K=36 < 64라 창당 64 사이클이 되어 픽셀당 1%가 는다). 재양자화가 채널별 단조 함수이므로 코드의 max는 실수 값의 max를 재양자화한 것과 같다. 정수 모델과 보드 드라이버가 같은 순서로 계산한다.

**학습.** Train400 중 20장을 검증용으로 떼고 380장으로 학습한다. 40×40(A) 또는 50×50(B 계열) 패치, 뒤집기·회전, MSE, Adam(초기 $10^{-3}$, 배치 128). 검증 PSNR이 6 epoch 동안 오르지 않으면 학습률을 반으로, 15 epoch 동안 오르지 않으면 종료한다. 검증 최고 모델을 쓴다. 체크포인트, 학습률 조정, 양자화 범위는 어느 것도 테스트셋(Set12, BSD68)으로 고르지 않았다. 데이터, 증강, 손실, 학습률 규칙은 모든 망이 같고 epoch당 픽셀 수도 같지만(52.4M 대 51.2M), epoch당 갱신 횟수는 A 256, B 계열 160이다. 최고 검증 epoch까지의 갱신은 A 2.7만, B 2.2만이었다. B-32 세 망(ReLU, 평균 결합, maxout)과 45채널 ReLU 망은 조기 종료 없이 같은 120 epoch(갱신 19,200회)을 학습하고 검증 최고 모델을 썼다. 처음 조기 종료로 학습했을 때 B-32(ReLU)는 epoch 26에 검증 PSNR이 무너진 뒤 15 epoch 안에 회복하지 못해 epoch 40(갱신 6,400)에 멈췄고 쓴 체크포인트는 epoch 25(갱신 0.4만; maxout은 1.8만)였으며, 그래서 예산을 맞춰 다시 학습했다. 시드는 B-32 ReLU와 maxout이 1·2·3, 평균 결합이 1·2이고 나머지 망은 1이다. 시드 1은 CPU에서, 시드 2·3과 45채널 망은 다른 PC

의 GPU(PyTorch 기본 설정)에서 학습했다. 가중치 초기값과 학습 패치·잡음은 시드만으로 정해져 장치가 달라도 같고, 다른 것은 부동소수점 연산이다(GPU 기본값에서는 합성곱이 TF32일 수 있어 정밀도도 다르다). 같은 시드의 첫 epoch 학습 손실은 CPU와 GPU에서 ReLU 0.07%, 평균 결합 0.13% 안에서 같았지만, maxout은 1.2% 달라 CPU에서 시드를 바꾼 차이(0.4%)보다 컸다. 연산 정밀도 차이로 학습 궤적이 갈라진 것과 맞는다.

**양자화와 양자화 인지 학습.** 정수 추론 [5]을 따른다. 가중치는 출력 채널별 대칭 INT8, 바이어스 INT32, 활성값은 층(맵)마다 스케일 하나를 갖는 7비트. 스케일은 먼저 학습 영상에서 맵마다 활성값 분포의 퍼센타일을 골라 정하고(범위 통계는 Train400의 매 10번째 40장, 퍼센타일 선택은 그와 겹치지 않는 다른 40장이며, Set12와 BSD68은 쓰지 않음), 이어서 **보드의 정수 연산을 float 안에서 그대로 재현한 모듈**(INT8 가중치 반올림, 7비트 활성값 반올림, 바이어스 단위, skip 입력의 스케일 비 접기, 재양자화 뒤의 maxout, 8비트 출력)을 straight-through 추정으로 15 epoch 미세조정한다(학습률 $10^{-4}$). 활성값 스케일은 학습 변수다(LSQ [6]). 검증은 양자화를 켠 채로 하므로 남는 모델이 곧 보드 모델이다. 같은 스케일을 주면 이 모듈과 정수 모델의 PSNR은 0.001 dB 안에서 같다. 이 방식 자체는 FPGA용 양자화 도구들이 쓰는 비트 정확 에뮬레이션과 같은 관행이며, 본 코어에 맞춘 것은 7비트 부호 없는 활성값, 재양자화 뒤의 maxout, skip 스케일 접기다.

## IV. 검증

**RTL.** UVM [11] 환경(AXI4-Lite·AXI4-Stream 에이전트, 모니터에서 예측하는 참조 모델 스코어보드, 형상·모드·데이터·흐름 커버리지, AXI 핸드셰이크 assertion 8개)을 Verilator 5.052에서 돌렸다. 테스트 4종을 빌드 7개(4값 변형 3개 포함)와 시드 2-3개로 실행해 UVM 1.2에서 69회, IEEE 1800.2-2020에서 46회 모두 통과했고(xsim은 돌리지 않았다), UVM 1.2 실행에서 출력 63,179개와 레지스터 읽기 11,419개를 비교했다(2020판 실행은 37,525개와 7,659개). 일부러 넣은 버그 15개(4값 경로 5개 포함)를 모두 검출했다. 형상 커버리지는 축소 빌드에서 89-100%, 보드 빌드에서는 17-29%다(4값 변형은 86-94%, 14-29%). 지시 테스트는 CIFAR-10 층 형상(K=27-1152, COUT=32-128)을 덮으며, 잡음 제거망의 형상(K=9, 36, 297, 324, 612 등)은 UVM이 아니라 보드에서 비트 단위로 검증했고(1값과 4값 모두 네 망의 모든 층), 4값 변형은 보드에 올리기 전에 층마다 실제 창 512개씩의 RTL 시뮬레이션(II절)으로도 확인했다.

**보드.** 층마다 보드 출력을 PC 정수 모델의 정답과 보드 쪽에서 값 하나하나 비교하고, 최종 PSNR이 정수 모델의 값과 같아야 통과한다. 본 논문의 모든 "보드" 수치는 이 검사를 통과한 것이다.

## V. 결과

**조건.** Set12(12장), BSD68 [7] (68장), 가우시안 잡음 σ=25. 모든 기법에 같은 잡음 영상(고정 시드)을 **클리핑하고 7비트로 양자화해서** 넣었다. 이 조건은 CNN에는 학습 분포이지만 BM3D에는 아니다. 클리핑 없는 float 잡음에서 BM3D는 Set12 +0.03, BSD68 +0.26 dB 높고(포화 픽셀이 많은 영상에 집중), BM3D의 σ를 학습 영상에서 다시 골라도 차이는 0.1 dB 이내다. 표에는 두 값을 모두 적는다. 화질은 PSNR과 함께 SSIM [27] (8비트 영상, 11×11 가우시안 창, σ=1.5)으로도 쟀다. SSIM은 모든 출력을 같은 잡음에서 다시 만들고, 사진별 PSNR이 저장된 값과 맞는지(정수 모델 $10^{-5}$ dB, BM3D·DnCNN-S $10^{-3}$ dB 안) 먼저 확인한 뒤 계산했다(`denoise/ssim_eval.py`). 같은 입력에서의 CNN 기준선으로 공개 DnCNN-S [2] 가중치(17층 × 64, 556k, float, 클리핑 없이 학습)를 평가했다. 정수 PSNR은 보드와 같은 정수 연산을 PC에서 수행한 값이며, 보드에서 실행한 일곱 모델(A, B, B+QAT, 조기 종료판 B-32+maxout과 그 QAT 모델, 같은 예산판 B-32 ReLU+QAT와 B-32+maxout+QAT)마다 Set12 05.png 한 장을 끝까지 실행해 전 층 비트 일치를 확인했다(표 II). 4값 변형으로도 네 망(A, B+QAT, 같은 예산판 B-32 ReLU+QAT와 B-32+maxout+QAT)을 같은 영상으로 실행해 전 층 비트 일치를 확인했다. B-32 비교는 시드 세 개로 하고, 시드가 하나인 망끼리의 0.1 dB 미만 차이는 해석하지 않는다.

*표 I. 평균 PSNR(dB)과 코어 비용. 정수 = 보드와 같은 정수 연산의 값. 사이클은 보정 모델이며 "실측"은 보드에서 잰 모델(05.png)이다. SSIM은 정수 모델(보드와 같은 8비트 출력)의 값이며, BM3D와 DnCNN-S는 float 출력을 8비트로 반올림해 계산했다. 마지막 열은 정수 모델이 BM3D(클리핑 입력)보다 PSNR이 높은 사진 수. B-32와 45 채널 행은 조기 종료 없이 120 epoch(갱신 19,200회)을 학습했고, +QAT 행은 여기에 QAT 15 epoch을 더했다*

*(QAT 단계의 시드는 모든 QAT 모델이 3). 시드 표시가 없는 B-32 행은 시드 1이고, "시드 1-3" 행은 세 시드의 평균(괄호는 범위)이다. 평균 결합 QAT 모델의 float 값은 뜻이 없어 적지 않는다(아래 QAT 항목).*

| 모델 | 사이클/px | 곱셈기 활용률 | 파라미터 | Set12 float / 정수 | BSD68 float / 정수 | SSIM Set12 / BSD68 | > BM3D |
|---|---|---|---|---|---|---|---|
| BM3D [1], 클리핑 입력 | — | — | — | 29.95 | 28.37 | 0.8504 / 0.7990 | — |
| BM3D, 클리핑 없음 | — | — | — | 29.98 | 28.62 | 0.8513 / 0.8037 | — |
| DnCNN-S [2], 같은 입력 (float) | 9.3k (모델) | — | 556k | 30.35 | 28.89 | 0.8594 / 0.8217 | — |
| A: 7층 × 32 (기준) | 1,775 (실측) | 41% | 47k | 29.87 / 29.83 | 28.83 / 28.80 | 0.8422 / 0.8088 | 6/12, 60/68 |
| B: s2d, 10층 × 64 | 1,323 (실측) | 88% | 300k | 30.14 / 30.10 | 29.01 / 29.00 | 0.8532 / 0.8187 | 10/12, 68/68 |
| **B + QAT** (화질) | 1,323 (실측) | 88% | 300k | 30.13 / **30.12** | 29.01 / **29.02** | **0.8545 / 0.8206** | **10/12, 68/68** |
| B-32: s2d, 10층 × 32, ReLU | 669 | 45% | 76k | 29.55 / 29.51 | 28.55 / 28.53 | 0.8335 / 0.7959 | 2/12, 49/68 |
| B-32 + QAT | 669 (실측) | 45% | 76k | 29.58 / 29.62 | 28.66 / 28.68 | 0.8400 / 0.8065 | 5/12, 57/68 |
| B-32 + QAT, 시드 1-3 | 669 | 45% | 76k | 29.79 / 29.82 (29.62-29.94) | 28.81 / 28.82 (28.68-28.91) | 0.8459 / 0.8122 | 5·8·7/12, 57·68·68/68 |
| B-32 + maxout | 675 | 87% | 152k | 30.02 / 29.86 | 28.94 / 28.80 | 0.8475 / 0.8115 | 6/12, 66/68 |
| **B-32 + maxout + QAT** (속도) | 675 (실측) | 87% | 152k | 30.00 / **30.00** | 28.95 / **28.94** | **0.8512 / 0.8171** | 9/12, **68/68** |
| B-32 + maxout + QAT, 시드 1-3 | 675 | 87% | 152k | 29.98 / 29.97 (29.91-30.00) | 28.93 / 28.92 (28.89-28.95) | 0.8501 / 0.8163 | 9·7·9/12, 68·68·68/68 |
| B-32 + 평균 결합 + QAT, 시드 2 | 675 | 87% | 152k | — / 29.82 | — / 28.82 | 0.8457 / 0.8115 | 6/12, 65/68 |
| 10층 × 45, ReLU + QAT (s2d) | 934 | 62% | 150k | 30.07 / 30.05 | 28.98 / 28.97 | 0.8516 / 0.8174 | 10/12, 68/68 |

- **화질 모델(B+QAT):** 클리핑 BM3D보다 Set12 +0.17, BSD68 +0.66 dB(10/12, 68/68장). 클리핑 없는 BM3D보다도 +0.14 / +0.40 dB(10/12, 68/68). 기준 망 A보다 1.34배 적은 사이클로 +0.29 / +0.22

dB(12/12, 68/68). SSIM으로도 클리핑 BM3D보다 +0.004 / +0.022(10/12, 67/68장), 클리핑 없는 BM3D보다 +0.003 / +0.017(10/12, 68/68장) 높다.

- **속도 모델(B-32+maxout+QAT):** A보다 **2.63배** 적은 사이클로 A보다 +0.17 / +0.14 dB(12/12, 64/68장) 높다. Set12는 BM3D 수준(클리핑 기준 +0.04 dB, 9/12장; 클리핑 없는 BM3D보다 +0.01 dB), BSD68은 클리핑 BM3D보다 +0.57, 클리핑 없는 BM3D보다 +0.32 dB(67/68). 같은 구조를 조기 종료로 학습한 이전 판(29.95 / 28.91 dB)과의 차이 +0.04 / +0.03 dB는 해석하지 않는다(0.1 dB 미만). 이 모델을 고른 이유는 점수가 아니라 아래 같은 예산 비교에 들어간 바로 그 모델이기 때문이며, 시드 2·3이 나오기 전에 정했다. 세 시드의 평균은 29.97 / 28.92 dB(범위 29.91-30.00 / 28.89-28.95)이고, 시드 1은 Set12에서 셋 중 가장 높고(시드 3보다 0.002 dB) BSD68에서 둘째다. 합치기 없는 B-32 ReLU+QAT도 시드 평균 29.82 / 28.82 dB로 A(29.83 / 28.80)와 같은 화질을 2.66배 적은 사이클에 낸다. 즉 space-to-depth로 더 깊은(10층) 망을 넣고도 같은 화질에서 사이클이 크게 준다. SSIM은 Set12에서 BM3D와 같은 수준(0.8512 대 클리핑 0.8504, 클리핑 없음 0.8513; 8/12, 7/12장)이고 BSD68에서 높으며(0.8171 대 0.7990 / 0.8037; 66/68, 63/68장), A보다는 80/80장 모두 높다(+0.009 / +0.008).
- **공짜 출력 채널의 효과 (시드 세 개):** 같은 예산(120 epoch + QAT 15 epoch)에서 정수 PSNR(Set12 / BSD68)은 maxout 망이 시드 1·2·3에서 30.00 / 28.94, 29.91 / 28.89, 29.99 / 28.95 dB, ReLU 망이 29.62 / 28.68, 29.94 / 28.91, 29.89 / 28.88 dB다. 사이클은 maxout이 1% 많다(보드 실측 1.01배; 첫 층). 평균은 maxout이 +0.15 / +0.10 dB 높지만, 시드별 차이는 +0.37 / +0.26, −0.03 / −0.02, +0.10 / +0.07 dB로 시드 2에서 뒤집혔다(시드 2는 ReLU가 80장 중 69장에서 높다). 사진별 차이의 표준오차는 모두 0.04 dB 이하라서 불확실성은 테스트 사진이 아니라 학습 실행에서 온다. 시드 2·3을 보기 전에 정한 규칙(모든 maxout 시드가 모든 ReLU 시드보다 두 테스트셋에서 높을 때만 '높다'고 쓴다)에 따라, maxout이 화질을 올린다고 주장하지 않는다. 시드 1의 큰 차이는 그 시드의 ReLU 망이 epoch 26에 무너지고(검증 27.93 → 24.27 dB) 학습률 규칙이 회복 중에 학습률을 네 번 반으로 줄인 탓이 컸다. 이런 붕괴(검증 PSNR이 그때까지의 최고보다 1 dB 넘게 떨어짐)는 120 epoch 실행 아홉 개 중 넷(ReLU 시드 1·3, maxout 시드 2, 평균 결합 시드 2)에서 망 종류와 상관없이 일어났고, 망마다 붕괴한 실행이 붕괴하지 않은 실행보다 모두 낮았다(평균 결합은 float Set12 기준). 즉 시드 간 퍼짐은 대부분 붕괴한 실행에서 생긴다. 학습률을 낮추거나 기울기 클리핑을 쓰면 줄 것으로 보이지만 시험하지 않았다. maxout 망은 파라미터가 2배이므로(76k 대 152k) 두 대조군을 함께 봤다. 같은 파라미터의 45채널 ReLU 망(150k, 934 사이클/px)은 30.05 / 28.97 dB로, 시드가 하나지만 32채널 실행 여섯 개 모두보다 높았다(가장 좋은 32채널 실행보다 +0.06 / +0.03 dB). 대신 사이클이 38% 더 든다. 같은 64 출력을 max 대신 평균으로 합치는 대조군(초기 가중치와 데이터까지 maxout과 같음)은 시드 1에서 float 검증 최고 28.549 dB, Set12 30.05 dB로 maxout(28.528, 30.02)과 같거나 약간 높았고, 시드 2에서 정수 29.82 / 28.82 dB로 maxout(29.91 / 28.89)보다 0.09 / 0.07 dB 낮았다. 두 시드 모두 0.1 dB 안이라 max와 평균의 차이는 해석하지 않는다(시드 1의 QAT 정수 값은 아직 없다). 곱셈기 활용률은 45% → 87%이고, 7층 32채널 망의 15 epoch 스크린에서는 maxout과 ReLU의 차이가 없었다(마지막 5 epoch 평균 +0.02 dB). 64채널 망에서는 출력 128이 K를 넘겨 공짜가 아니다. 처음의 조기 종료 비교(+0.54 / +0.35 dB)는 ReLU 망의 체크포인트가 갱신 0.4만인 비교라 쓰지 않는다.
- **QAT의 효과:** 시드 1에서 학습 뒤 양자화(PTQ)의 정수 손실은 maxout 망 0.16 / 0.14 dB, ReLU 망 0.05 / 0.02 dB였고(Set12 / BSD68), ReLU·maxout 시드 2·3과 45채널 망까지 보면 0.02-0.25 dB다(평균 결합 시드 2는 0.29 / 0.17 dB). QAT 뒤에는 ReLU·maxout·45채널 정수 모델이 모두 float의 0.04 dB 안이다(시드 1 ReLU 망은 정수가 float보다 0.04 / 0.02 dB 높다). 시드 1의 정수 PSNR 이득은 maxout 망 +0.14 / +0.14 dB, ReLU 망 +0.12 / +0.15 dB(둘 다 80/80장)로 비슷하고 B는 +0.02 / +0.03 dB다. 평균 결합 QAT 모델의 float 값(양자화를 끈 망)은 활성값 클리핑까지 꺼져 BSD68 한 장이 8.74 dB로 무너지므로(평균 27.78 dB) 비교 기준이 되지 못한다. 클리핑만 남기면 28.75 dB로 정수 값(28.82)과 가깝고,

보드가 계산하는 정수 값과는 상관없다. QAT는 학습률 $10^{-4}$의 추가 15 epoch이기도 해서, 120 epoch 끝에 아직 오르던 ReLU 망은 float 점수도 +0.03 / +0.11 dB 올랐다.

- **SSIM과 PSNR:** 정수 모델 열네 개의 SSIM 순위는 PSNR 순위와 거의 같고(순위 상관 0.99), 자리가 바뀐 것은 PSNR 차이가 0.03 dB 미만인 모델끼리뿐이다. maxout 대 ReLU의 SSIM 차이도 PSNR처럼 시드마다 엇갈린다(시드 1·2·3에서 +0.011 / +0.011, −0.001 / −0.001, +0.003 / +0.002). BM3D보다 높은 사진 수는 모든 모델에서 SSIM이 PSNR보다 같거나 적고(화질 모델 BSD68 68 → 67장, A 60 → 49장, B-32 ReLU+QAT 57 → 45장), 그 차이는 대체로 약한 모델일수록 크다. 즉 SSIM에서는 BM3D가 상대적으로 강하다. 그래도 본문의 BM3D 비교(화질 모델은 두 테스트셋에서 높고, 속도 모델은 Set12에서 같은 수준, BSD68에서 높다)는 SSIM에서도 그대로 성립한다.
- **같은 입력의 DnCNN-S:** float 556k 모델이 30.35 / 28.89 dB(클리핑 없는 입력에서는 30.44 / 29.24로 문헌 [2]와 같다). 본 화질 모델(정수, 300k)은 Set12에서 0.2 dB 낮고 BSD68에서 0.1 dB 높은데, 후자는 DnCNN-S가 클리핑 없이 학습된 탓이므로 이득으로 세지 않는다. SSIM(0.8594 / 0.8217)으로는 화질 모델이 두 테스트셋 모두 약간 낮다(−0.005 / −0.001; BSD68에서 높은 사진 19/68장). DnCNN-S 형상(K=576, 17층)을 이 코어에 올리면 사이클 모델로 약 9.3k 사이클/px, 화질 모델의 7.0배다.

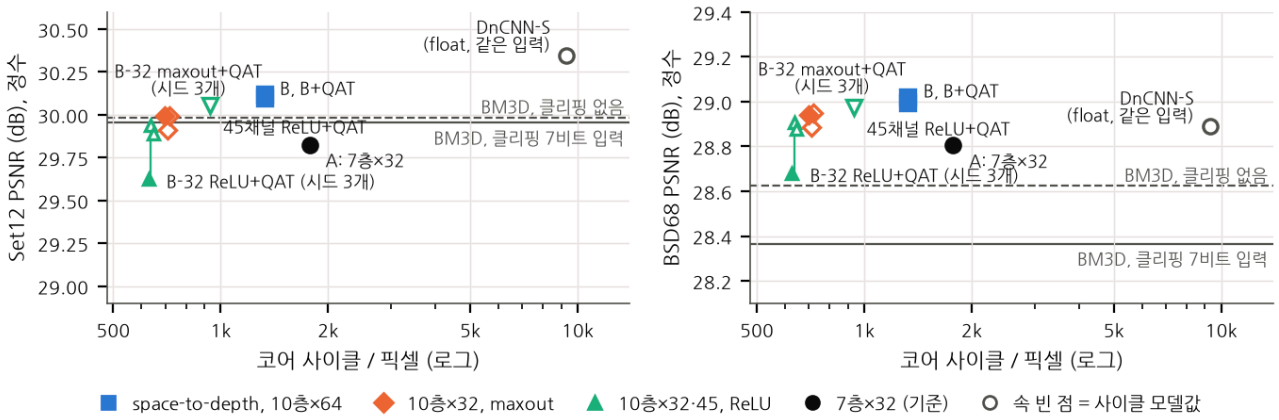



*그림 2. 코어 사이클/픽셀 대 정수 PSNR (왼쪽 Set12, 오른쪽 BSD68). 채운 점은 보드에서 잰 사이클, 속 빈 점은 사이클 모델값. 가로선은 BM3D(실선: 논문의 클리핑 7비트 입력, 점선: 클리핑 없음). B-32 점은 같은 예산 QAT 모델의 시드 세 개이고(세로선은 범위), 두 망 모두 약 670 사이클이지만 점이 겹치지 않게 x를 조금씩(최대 7%) 옮겨 그렸다. maxout의 세 시드는 ReLU 범위의 위쪽 끝과 겹치고, 같은 파라미터의 45채널 ReLU 망은 38% 더 많은 사이클에서 그 위에 있다. space-to-depth 망 B는 기준 망 A의 왼쪽 위에 있다.*

*표 II. 보드 실측 (Set12 05.png, 256×256, 모든 층 불일치 0). 코어 시간 = 사이클 / 클럭. 벽시계는 JTAG 전송과 보드 쪽 검증을 포함한 하네스 시간.*

| 모델 | 클럭 | 층 | K | COUT | 사이클/창 (모델→실측) | 총 사이클 | 코어 시간 | 벽시계 | 05.png PSNR |
|---|---|---|---|---|---|---|---|---|---|
| B + QAT | 50 MHz | conv1 / conv2-9 / conv10 | 36 / 576 / 612 | 64 / 64 / 4 | 36→64 / 576→577 / 612→613 | 86,721,997 (1,323/px) | 1.73 s | 600 s | 30.06 |
| B-32 + maxout + QAT | 50 MHz | conv1 / conv2-9 / conv10 | 36 / 288 / 324 | 64 / **64** / 4 | 36→64 / 288→**289** / 324→325 | 44,254,293 (675/px) | 0.89 s | 408 s | 29.94 |
| B-32 ReLU + QAT | 50 MHz | conv1 / conv2-9 / conv10 | 36 / 288 / 324 | 32 / 32 / 4 | 36→37 / 288→289 / 324→325 | 43,811,412 (668.5/px) | 0.88 s | 292 s | 29.54 |
| B-32 + maxout + QAT (조기 종료 학습판) | 50 / 100 MHz | 같음 | | | 같음 | 44,254,269 (두 클럭 같음) | 0.89 / 0.44 s | 371 / 399 s | 29.87 |
| A | 50 MHz | conv1 / conv2-6 / conv7 | 9 / 288 / 297 | 32 / **32** / 1 | 9→32 / 288→**289** / 297→298 | 116,326,743 (1,775/px) | 2.33 s | 701 s | 29.83 |
| B + QAT, 4값 | 50 MHz | 같음 | 같음 | 같음 | 64 / 평균 258-394 / 154 | 44,789,776 (683.4/px) | 0.90 s | 475 s | 30.06 |
| B-32 + maxout + QAT, 4값 | 50 MHz | 같음 | 같음 | 같음 | 64 / 평균 207-262 / 82 | 32,982,254 (503.3/px) | 0.66 s | 359 s | 29.94 |
| B-32 ReLU + QAT, 4값 | 50 MHz | 같음 | 같음 | 같음 | 32 / 평균 79-95 / 82 | 13,289,042 (202.8/px) | 0.27 s | 315 s | 29.54 |
| A, 4값 | 50 MHz | 같음 | 같음 | 같음 | 32 / 평균 84-104 / 76 | 37,520,551 (572.5/px) | 0.75 s | 585 s | 29.83 |

- 1값 행에서 maxout 층(K=288, 출력 64)은 A와 같은 폭 ReLU 망의 K=288, 출력 32 층과 똑같이 **289 사이클/창**이다. 1값 포트에서 출력을 2배로 늘린 비용이 0임을 보드가 보여 준다(4값 행에서는 망 전체로 2.48배, 아래). 첫 층(K=36)만 출력을 내보내는 시간 때문에 64 사이클(ReLU 망은 37)이 걸려, 망 전체로는 ReLU 망의 1.01배(44,254,293 대 43,811,412 사이클)다.
- 1값 행에서 보정 모델과 실측의 차이는 네 망(A, B, B-32 maxout, B-32 ReLU) 모두 0.003% 이내이며, 그 차이는 층 실행마다 붙는 수십~수백 사이클의 오버헤드다(망 전체로 343~1,485 사이클). 100 MHz에

서 사이클 수는 50 MHz와 같다. 같은 구조라도 가중치가 다르면 sparse 모드의 층별 사이클이 몇 사이클 달라진다(조기 종료판과 같은 예산판 B-32+maxout+QAT: 44,254,269 대 44,254,293).

- '4값' 행은 사이클당 4값을 받는 변형(II절)이다. PC가 창과 가중치를 4의 배수 길이로 0 채움한다. 은닉층은 sparse 모드에서 계산에 묶여 창마다 사이클이 다르므로 층별 평균의 범위를 적었다. 첫 층과 마지막 층은 창마다 일정하고(출력 COUT 또는 적재 ⌈K/4⌉+1) 모델과 같다. 네 망의 총 사이클은 비트스트림 전에 커밋한 예측보다 337, 1,372, 794, 586 사이클(A, B, B-32 maxout, B-32 ReLU; 0.0009-0.0044%) 많았다. B ÷ A는 1값의 0.75에서 1.19로, maxout ÷ ReLU는 1.01에서 2.48로 바뀌었다.
- 절대 처리량은 낮다. 50 MHz, 곱셈기 64개가 실제로 수행하는 연산은 최대 6.4 GOPS(곱셈-누산 하나를 2연산으로 셈; 1값에서 B 망은 dense 연산 기준 실효 5.7 GOPS)이며 256×256 한 장에 코어만 0.9-2.3 s다. 4값 변형은 0.27-0.90 s인데, sparse 모드가 0인 활성값의 곱셈을 건너뛰므로 dense 연산으로 환산한 실효는 이 최대를 넘는다(B 망 약 11 GOPS). 같은 망을 PC CPU(Xeon 4코어, float32 PyTorch)는 30-38 ms에 처리한다. 같은 칩의 ARM 프로세서와는 비교하지 않았다. 이 논문은 처리량이 아니라 주어진 코어에서 망을 고르는 근거를 다룬다. 전력을 재지 않았으므로 GOPS/W는 보고하지 않는다.

*표 III. 입력 구조별 코어 사이클/픽셀. Set12 05.png의 모든 창을 센 값이며(sparse 모드, 보드와 같음) 괄호는 dense 상한이다. 창당 비용은 max(적재 + 1, ⌈COUT/P⌉ × 그룹당 비용, COUT)이고, 적재는 ⌈K/w⌉ 또는 라인 버퍼의 C_in, 그룹당 비용은 뱅크별 0 아닌 탭 수의 최댓값(dense는 ⌈K/4⌉)이다(`denoise/wider_ports.py`). 1값은 보드 실측이다. 4값의 sparse 값은 같은 식으로 센 값이고, 보드 실측(표 II)과 표의 자릿수까지 같다(괄호의 dense 값은 보드에서 돌리지 않았다). 2값과 라인 버퍼는 예측이다. 라인 버퍼는 창마다 새 입력 픽셀 하나(C_in개 값)가 들어오는 구조다.*

| 망 | 1값/사이클 (보드) | 2값/사이클 | 4값/사이클 (보드) | 라인 버퍼 |
|---|---|---|---|---|
| A: 7층 × 32 | 1,775 | 907 (907) | 573 (828) | 538 (827) |
| B: s2d, 10층 × 64 | 1,323 | 741 (1,245) | 683 (1,207) | 665 (1,206) |
| B-32 ReLU | 669 | 339 (339) | 203 (317) | 191 (316) |
| B-32 maxout | 675 | 524 (633) | 503 (613) | 500 (612) |
| B ÷ A | 0.75 | 0.82 (1.37) | 1.19 (1.46) | 1.23 (1.46) |
| maxout ÷ ReLU | 1.01 | 1.55 (1.87) | 2.48 (1.94) | 2.61 (1.94) |

sparse 모드에서 합치기는 2값부터 공짜가 아니고(maxout 망은 입력에 0이 적어 건너뛸 곱셈이 적다), B는 4값과 라인 버퍼에서 A보다 느려진다. dense 상한으로는 2값만 돼도 두 이점이 모두 사라진다. 다만 B-32 maxout은 어느 구조에서도 A보다 빠르다. 입력 포트(코어의 적재 경로와 래퍼)를 넓히는 하드웨어 개선과 망 선택은 어느 하나를 고르는 관계가 아니다. 포트를 넓히면 모든 망이 빨라지고(보드에서 A 3.10배, B 1.94배, B-32 maxout 1.34배, B-32 ReLU 3.30배), 그때 이기는 망이 달라진다는 것이 이 표의 요점이다. 4값 열은 보드에서 쟀고, 2값 포트와 라인 버퍼는 만들지 않았다.

**같은 사이클의 다른 설계: 2단계 U-Net.** 리샘플링이 공짜라는 성질로 깊이를 1/16 해상도 층에 두는 2단계 U-Net(보정 모델 1,303 사이클/px, 667k 파라미터)도 구현해 30 epoch 스크린했다. 검증 PSNR 27.89 dB로 같은 epoch의 B(28.14)에 0.25 dB 뒤졌으나, 갱신 횟수(100 대 160)와 배치·패치 크기가 달랐고 두 곡선 모두 상승 중이어서 결론은 아니며 보류했다. 공짜 집합 안의 선택도 결국 표에서 가려야 한다.

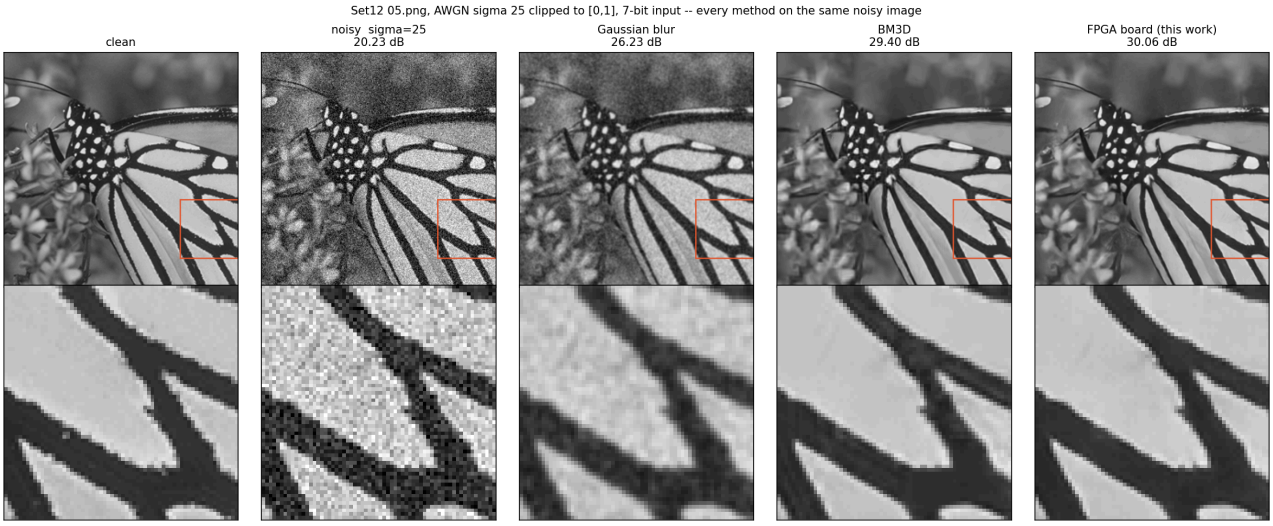



*그림 3. Set12 05.png. 왼쪽부터 원본, 잡음(20.23 dB), 가우시안 필터(26.23 dB), BM3D(29.40 dB), FPGA 보드 출력(B+QAT, 30.06 dB). 아래 줄은 확대 영역이다.*

## VI. 결론

곱셈기 64개의 소형 정수 가속기에서, 실측과 맞는 사이클 모델이 병목이 곱셈기가 아니라 입력 포트임을 보였다. 그 포트 폭에서는 연산량으로 손해인 두 설계의 비용이 작았다. space-to-depth는 6.4배 큰 망을 25% 적은 사이클에 넣어 화질도 올렸고, 같은 32채널 망은 시드 평균으로 기준 망과 같은 화질을 2.66배 적은 사이클에 냈다. 은닉층 출력을 2배로 늘려 두 개씩 합치는 설계는 은닉층의 사이클을 바꾸지 않고 파라미터를 2배로 늘렸지만, 같은 폭의 ReLU 망보다 높은 화질(시드 평균 +0.15 / +0.10 dB)은 시드 세 개 중 하나에서 뒤집혀 확인되지 않았다. 양자화 인지 학습까지 더해, 같은 입력의 BM3D를 두 테스트셋에서 PSNR과 SSIM 모두 넘는 정수 모델(30.12 / 29.02 dB, SSIM 0.8545 / 0.8206)과 기준보다 2.63배 적은 사이클로 BM3D 수준인 모델(30.00 / 28.94 dB)을 얻었고, 두 모델 모두 보드에서 한 장을 끝까지 실행해 전 층 비트 일치를 확인했다. 입력 포트를 사이클당 4값으로 넓힌 변형을 보드에서 돌리자, 미리 커밋한 예측대로 B는 A보다 느려지고(1.19배) 합치기는 공짜가 아니게 되어(ReLU 망의 2.48배), 고를 망이 병목을 따라 바뀜을 측정으로 확인했다. 코드, 정수 모델, 사진별 PSNR, 보드 기록은 저장소에 있다.

**어디까지 일반적인가.** 구체적인 규칙(출력 채널 2배가 공짜, 64채널 망 B가 A보다 빠름)은 PC가 만든 창을 사이클당 값 하나로 받는 이 입력 구조에서만 성립하며, 표 III처럼 입력이 2값/사이클이면 합치기가, 4값/사이클이나 라인 버퍼면 B의 이점도 사라진다(창을 센 sparse 값 기준이며 4값은 보드에서 쟀다; dense 상한으로는 2값에서 둘 다). 일반화되는 것은 두 가지다. 첫째는 판정 기준이다. 추가 사이클 없이 늘릴 수 있는 출력 채널은 대략 곱셈기 수를 사이클당 입력 값 수로 나눈 값(P·T/w)이다. 곱셈기 256개에 4값/사이클인 가속기라면 256/4 = 64이므로 32채널 망의 출력 2배는 다시 공짜가 되고, 이 코어를 2값/사이클로 넓히면 공짜가 아니다. 둘째는 방법이다. 병목을 재고, 그것을 맞히는 사이클 모델을 세우고, 공짜인 자원을 쓰는 망을 고른다. 병목에 맞춰 설계를 고르는 일은 다른 플랫폼에서도 나타난다. TPU·GPU에서는 space-to-depth와, 일부 단계에서는 연산량이 더 많은 일반 합성곱(fused MBConv)이 활용률을 올려 오히려 빨랐고 [13], [22], 메모리 대역에 묶인 LLM 추론에서는 노는 연산으로 토큰 여러 개를 한 번에 검증하는 추측 디코딩 [23]이 쓰이며, GPU에서는 잰 지연의 계단에 맞춰 채널 수를 고르고 [25], 시스톨릭 배열에서는 지연이 가장 짧은 망(그룹당 채널 수)이 배열 크기에 따라 달라진다 [26]. MCU에서는 노는 자원이 아니라 메모리 용량이라는 다른 병목에 맞춰 망과 실행 순서를 함께 설계한다 [24]. 이 코어의 4값 포트에서는 P·T/w의 바탕인 사이클 계수를 보드가 확인했고(예측과 0.005% 안), 2값 포트와 라인 버퍼에서는 아직 예측이다.

**한계.** 사이클은 공짜지만 에너지는 아니다. maxout 망은 dense 곱셈이 ReLU 망의 2배이고, sparse 모드에서 실제로 수행한 0이 아닌 곱셈은 3.0배였다(보드 기록, 05.png; 은닉층 입력의 0이 아닌 비율 0.64-0.84 대 0.37-0.55). 동적 전력이 늘 수 있지만 측정하지 않았다. 처리 시간은 코어 사이클만 세었고 end-to-end 시간은 JTAG 하네스 기준이다. 잡음은 σ=25 하나이고, 시드 세 개는 B-32 ReLU·maxout에만 있고(평균 결합은 두 개, 그중 시드 1은 float 값만; 나머지 망은 하나), 흑백 영상만 다뤘다. 규칙은 현재 입력 구조(포트 폭, host im2col)에 조건부이며, 2값 포트와 라인 버퍼, 4값 포트의 dense 모드, 1값 dense 모드의 COUT > 64는 보드에서 재지 않았다. 모든 PSNR은 클리핑한 7비트 입력 조건이다.

**사사.** 본 연구는 과학기술정보통신부 및 정보통신기획평가원의 "SW중심대학사업" 지원을 받아 수행되었습니다.
〔저자 결정, 안〕 코드와 초안 작성에 AI 코딩 도구(Claude Code)를 사용했다. 설계 판단, 실험 설계, 보드 실행과 검증은 저자가 했다.

## 참고문헌

[1] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, "Image denoising by sparse 3-D transform-domain collaborative filtering," *IEEE Trans. Image Process.*, vol. 16, no. 8, pp. 2080-2095, 2007.

[2] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142-3155, 2017.

[3] K. Zhang, W. Zuo, and L. Zhang, "FFDNet: Toward a fast and flexible solution for CNN-based image denoising," *IEEE Trans. Image Process.*, vol. 27, no. 9, pp. 4608-4622, 2018.

[4] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," in *Proc. ICML*, 2013.

[5] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018.

[6] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in *Proc. ICLR*, 2020.

[7] D. Martin, C. Fowlkes, D. Tal, and J. Malik, "A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics," in *Proc. ICCV*, 2001.

[8] Y. Ma, Y. Cao, S. Vrudhula, and J.-s. Seo, "Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks," in *Proc. FPGA*, 2017.

[9] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. ECCV*, 2018.

[10] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 11, pp. 2884-2896, 2018.

[11] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020.

[12] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[13] S. Li, M. Tan, R. Pang, A. Li, L. Cheng, Q. V. Le, and N. P. Jouppi, "Searching for fast model families on datacenter accelerators," in *Proc. CVPR*, 2021, pp. 8085–8095.

[14] Z. Du, J. Liu, J. Tang, and G. Wu, "Anchor-based plain net for mobile image super-resolution," in *Proc. CVPR Workshops*, 2021, pp. 2494–2502.

[15] C.-T. Huang, Y.-C. Ding, H.-C. Wang, C.-W. Weng, K.-P. Lin, L.-W. Wang, and L.-D. Chen, "eCNN: A block-based and highly-parallel CNN accelerator for edge inference," in *Proc. MICRO*, 2019, pp. 182–195.

[16] J.-S. Choi and M. Kim, "Single image super-resolution using lightweight CNN with maxout units," in *Proc. ACCV*, 2018 (LNCS, Springer, 2019), pp. 471–487.

[17] S. Gu, W. Li, L. Van Gool, and R. Timofte, "Fast image restoration with multi-bin trainable linear units," in *Proc. ICCV*, 2019, pp. 4190–4199.

[18] W. Kang et al., "Improvement and hardware design of image denoising algorithm based on deep learning," in *Proc. Int. Conf. Integrated Circuits and Microsystems (ICICM)*, 2024, pp. 671–676.

[19] L. Tu, Y. Gu, X. Huang, and X. Wu, "Lightweight design of image denoising based on TNet mini," in *Proc. Int. Symp. Artificial Intelligence and Intelligent Manufacturing (AIIM)*, 2024, pp. 646–649.

[20] C. Li, C. Bled, R. Fernandez, and S. Shanker, "ReTiDe: Real-time denoising for energy-efficient motion picture processing with FPGAs," in *Proc. ACM SIGGRAPH European Conf. Visual Media Production (CVMP)*, 2025.

[21] D. Wang, K. Xu, and D. Jiang, "PipeCNN: An OpenCL-based open-source FPGA accelerator for convolution neural networks," in *Proc. Int. Conf. Field-Programmable Technology (FPT)*, 2017.

[22] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. ICML*, 2021, pp. 10096–10106.

[23] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. ICML*, 2023, pp. 19274–19286.

[24] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Proc. NeurIPS*, 2020, pp. 11711–11722.

[25] M. Shen, H. Yin, P. Molchanov, L. Mao, J. Liu, and J. M. Alvarez, "Structural pruning via latency-saliency knapsack," in *Proc. NeurIPS*, 2022 (HALP).

[26] N. K. Jha, S. Ravishankar, S. Mittal, A. Kaushik, D. Mandal, and M. Chandra, "DRACO: Co-optimizing hardware utilization, and performance of DNNs on systolic accelerator," in *Proc. ISVLSI*, 2020.

[27] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.